Algorithms for Encoding Interstellar Messages

Michael Matessa[a]

[a]*METI International, 100 Pine St, Suite 1250, San Francisco, 94111, CA, USA*

Abstract

Previous work has developed algorithms for decoding the meaning of an interstellar message from a distant civilization (Matessa, 2022). What about the inverse problem: if you have a concept like the mass of an object that you want to discuss, how can you create a message with that concept that can be easily decoded? This paper describes algorithms for creating a sequence of symbols that introduce concepts leading from basic mathematics to a target concept. The algorithms work with a data structure containing an expression form that relates concepts, a precondition of concepts that have been introduced previously in the message, and a postcondition of concepts that should be understood after the expression form is presented. These algorithms have been implemented in software, and results are shown for creating a message given a target concept. Messages that are created by the algorithms in this paper can be decoded by the algorithms in Matessa (2022). By developing algorithms, it is hoped that message encoding and decoding can grow from an art done by individuals to a science done with algorithms.

Keywords: Messaging Extraterrestrial Intelligence, METI, Interstellar messages

1. Introduction

If you have a concept that you want to discuss in an interstellar message, how can you create a message that can be easily decoded? When decoding a message, the receiver uses patterns to try to deduce the meaning [1, 2]. Matessa [3] formalizes the deduction process with algorithms. This paper describes algorithms for creating a sequence of symbols that introduce concepts that proceed from basic mathematics to a target concept. The patterns in this sequence can be decoded by the algorithms in Matessa [3]. These algorithms take advantage of the simplifying assumption that messages consist of

a linear sequence of symbols grouped into expressions. This type of message includes linear, Lincos-type messages [4, 5, 6] but excludes two-dimensional, Arecibo-type messages [7, 8, 9, 10]. By developing algorithms, it is hoped that message encoding can grow from an art done by individuals to a science done with algorithms.

Previous work on algorithms for encoding messages has mostly focused on taking symbolic messages encoded by hand and converting the symbolic level into binary or other forms that can be given to radio telescopes to transmit [5, 8]. At the symbolic level, Ollongren [11] describes a logic system to annotate previously-created messages. Ollongren and Vakoch [12] use an automated proof assistant to check the correctness of logical expressions in a created message. McConnell [2] describes algorithms that could be used to implement computer code and simulations in messages to describe concepts such as orbits. Fitzpatrick [13, 14] developed a message as computer code called Cosmic OS. The goal of Cosmic OS is to communicate enough structure to simulate a simple MUD (multi-user dungeon) and to use the interactions between locations, objects, and characters as an alternative to the "morality plays" that describe concepts in Lincos [4].

2. Algorithms for Encoding Messages

The algorithms in this paper differ from those described above in that they create a sequence of expression forms that facilitate the understanding of concepts, represented by symbols in the expressions. Simple concepts are presented first, and more complex concepts are built on earlier parts of the message. Messages are assumed to refer to concepts in mathematics and physics that would be similar for all civilizations in our universe.

The algorithms work with a data structure containing an expression form that relates concepts, a precondition of concepts that have been introduced previously in the message, and a postcondition of concepts that should be understood after the expression form is presented. These data structures can be considered as rules: given the understanding of concepts in the precondition list, show these expressions in order to facilitate the understanding of the concepts in the postcondition list. Given a target concept, a backwards chain is created so that the target is a postcondition of a rule, preconditions of that rule are postconditions of previous rules, back to a rule that does not have preconditions. The expression forms in rules were chosen to be able to be decoded by the pattern-finding algorithms in Matessa [3]. The forms in-

troduce the concepts of Equality, Counting, Function, Base, Approximation, Constants, and Planck Patterns.

The algorithms deal with a set of rules that rely on each other in a specific order. The find-all-rules algorithm starts with a single rule containing a target concept in its postcondition and explores all the rules it depends on, building a list that reflects this dependency order. It does this by checking if the postconditions of one rule matches the preconditions of another rule, and if that new rule hasn't been seen yet. If both conditions are true, the new rule and its dependency level are added to a list. By following these dependencies recursively, find-all-rules creates a sorted list where rules with no dependencies come first. The show-forms algorithm then takes this dependency list and goes through each rule in the order established by find-all-rules. It checks if the rule has been encountered before, and if not, puts its expression forms into a message. A list of all the used rules is also stored. In short, find-all-rules figures out the order in which the rules can be applied based on their dependencies, and show-forms then uses this order to put the expression forms for each rule into a message.

The variable lvl is used by the algorithms to track the depth within the dependency tree of rules. Here's how it's used:

- Initialization: In the find-all-rules algorithm, lvl is initialized to 0 when the algorithm starts exploring rules.
- Tracking Depth: Inside the find-pre-rules algorithm, lvl is incremented by 1 whenever a new level of dependency is encountered. This happens when the algorithm recursively calls itself to explore the pre-requisites (dependencies) of a rule. So, the value of lvl increases as the function dives deeper into the dependency tree.
- Adding to rule-lis: When a new rule is added to the rule-lis (list of rules with their dependency order), the corresponding lvl value is also added alongside the rule. This helps maintain the dependency order because rules with a higher lvl (farther from the target concept) have fewer dependencies.

In summary, lvl acts like a counter that keeps track of how many levels deep a specific rule is within the dependency tree. This information is then used to sort the rules in find-all-rules based on their dependencies.

The following sections describe how the Equality, Counting, Function, Base, Approximation, Constants, and Planck Patterns algorithms from Matessa [3] were used to create the rules and expression forms used in this work.

2.1. Equality

Mathematical equality is a powerful function that enables the presentation of new symbol meanings and their equivalence to previously established symbol meanings. For example, once the symbols for numbers have been established, symbols for mathematical functions such as addition and multiplication can be introduced. In the reflexive property of equality, an equals sign shows that a value is equal to itself. This form requires no prerequisite knowledge of what the values are, and so can be used to bootstrap further discussion using the equals sign. The Equality algorithm in Matessa [3] uses symbol patterns in expressions showing the reflexive property to determine what symbol maps onto the equals sign. For example, after seeing the expressions "s01 s10 s01" and "s03 s10 s03", the algorithm maps s10 to "=". The Equality rule in this work introduces the concept of equality with expressions like "1 = 1" and "3 = 3". The rule has no precondition and a postcondition of Equality, representing the understanding that the symbol "=" represents equality.

2.2. Counting

Natural numbers are easily described with multiple objects. The Counting algorithm in Matessa [3] uses patterns of repeating symbols to assign number values to symbols. For instance, given a symbol mapping of s10 to "=" and expressions "s03 s10 s11 s12 s12 s12 s13" and "s04 s10 s11 s12 s12 s12 s12 s13", that algorithm maps the symbols s03 and s04 to "3" and "4", and the symbols s11 and s13 to list delimiters [" and "]". The Counting rule in this work contains expressions like "2 = [o o]" and "3 = [o o o]". It has a precondition Equality, and a postcondition Numbers.

2.3. Function

The meanings of symbols for mathematical functions can be deduced by determining which functions are logically consistent with the numbers found in the expression. The Function algorithm in Matessa [3] determines these potential meanings for binary operators by iterating through a list of operators and calculating which are logically consistent. The Function rules

in this work provide expressions that uniquely limit operators. For example, in the expression "0 s14 0 = 0", s14 could potentially be "+", "-", or "*". The Function rules do not use these kinds of expressions, but instead use expressions like "4 + 5 = 9", which uniquely limits the meaning of the "+" symbol to be addition. Rules for different functions contain expressions that uniquely identify the functions related to numbers and contain that function as a postcondition.

2.4. Base

Although humans tend to use base ten for mathematics, cultures have used other bases, such as the Babylonians using base sixty. An alien species with two hands and six fingers on each (like the Twelve-toes from Schoolhouse Rock) might use base twelve. In this work, base ten is used for numbers with multiple digits. The Base rule has a precondition of knowing the symbol mappings for Equality, Numbers, +, -, *, and /. It has expressions such as "5 + 5 = 1 0" and a postcondition of Base10.

2.5. Approximation

Approximation is a useful concept for describing repeating numbers, irrational numbers, and physics ratios. The Approximation rule has preconditions of Number, /, and Base10. It has expressions such as "1 / 3 approx 0 . 3 3 3" and a postcondition of Approximation.

2.6. Constants

Dimensionless quantities such as pi and e have the same value in any culture and can be used in pattern matching of more complex physics equations. Constants rules have expressions for these values. For example, the Pi rule has the expressions "pi approx 2 2 / 7" and "pi approx 3 3 3 / 1 0 6", with the postcondtion "pi". One way to bridge mathematics and physics in an interstellar message is to use unitless physics ratio constants, which do not depend on knowing unit definitions such as pound or kilogram. For example, the ratio of the mass of a proton to the mass of an electron is approximately 1836, and the ratio of the neutron mass to the electron mass is approximately 1839. Other Constants rules use these ratios to define the meanings of the symbols. For example, the Mass-Ratio rule has the expression "mass [proton] / mass [electron] approx 1 8 3 6" with postcondition Mass, Proton, and Electron.

2.7. Planck Patterns

Another way to discuss measurements in a message is to use a system of measurement that does not depend on previously defined Earth standards. One such system is the Planck unit system. Planck units are defined exclusively in terms of four universal physical constants: c, the speed of light in a vacuum; G, the gravitational constant; h-bar, the reduced Planck constant; and kB, the Boltzmann constant. Figure 1 gives the definitions of Planck length, Planck mass, and Planck time:

Name	Expression
Planck length	$l_{\mathrm{P}} = \sqrt{\frac{\hbar G}{c^3}}$
Planck mass	$m_{\mathrm{P}} = \sqrt{\frac{\hbar c}{G}}$
Planck time	$t_{\mathrm{P}} = \sqrt{\frac{\hbar G}{c^5}}$

Figure 1: Mathematical definitions of Planck units

The Planck unit system provides patterns that can be used to determine the meanings of individual symbols. The Planck rule has expressions such as "planck [length] = sqrt [[hbar * G] / [c ^ 3]]" and postconditions of Length, Mass, and Time.

These patterns are used by the Planck algorithm in Matessa [3] as follows: The Planck algorithm is given expressions with symbols for basic mathematics that have been previously decoded, such as s31 [s32] = sqrt [[s29 * s33] / [s15 ^ 3]] and adds meanings for symbols consistent with the pattern into hypothesis lists. For example, after given the expression above the hypothesis lists would contain s31=(planck, length), s32=(planck, length), s29=(h-bar, G), s33=(h-bar, G), and s15=(c). Given the expression s31 [s23] = sqrt [[s29 * s15] / s33] the hypothesis lists would contain s31=(planck, mass), s32=(planck, mass), s29=(h-bar, c), s15=(h-bar, c), and s33=(G). Meta-algorithms derive logical consistency between these hypotheses and

give hypothesis lists of s31=(planck), s32=(length), s23=(mass), s29=(h-bar), s33=(G), and s15=(c).

A full list of rules with their preconditions and postconditions can be seen in Figure 2. Note how with a chaining of rules, a precondition of one rule can be satisfied by the postcondition of a previous rule.

Equality Rule
Pre: none
Post: Equality

Counting Rule
Pre: Equality
Post: Numbers

Function Rule
Pre: Equality, Numbers
Post: {Addition, Subtraction, Multiplication, Division, Exponent, SquareRoot}

Base Rule
Pre: Equality, Numbers, Addition, Subtraction, Multiplication, Division
Post: Base10

Approximation Rule
Pre: Numbers, Division, Base10
Post: Approximation

Constants Rule
Pre: Numbers, Divison, Base10, Approximation
Post: {pi, mass, proton, electron}

Planck Rule
Pre: Equality, SquareRoot, Multiplication, Division, Exponent
Post: Length, Mass, Time

Figure 2: Rules with pre- and post- conditions

3. Implemented System

The algorithms described above have been implemented in the Lisp programming language, and the code is available in Github at https://github.com/mmatessa/encoding-interstellar. As an example, this code was given the

target concept Length and encoded a message, shown in Appendix A. This message was given to the decoding algorithm code from Matessa [3] (available at at https://github.com/mmatessa/encoding-interstellar) and the message was successfully decoded. To do this, the symbols in the message were replaced by generic symbols S01, S02, etc. as seen in Appendix B. The decoding algorithm code was then able to map the symbols to the meanings, as seen in Appendix C.

4. Conclusions

This paper presents algorithms that can be used to encode an interstellar message to a distant civilization. As hypothesized in Matessa [3], the algorithms developed there for decoding messages are used here to help in the creation of messages. Expression forms were developed based on those algorithms, and pre- and post- conditions of expression forms are chained backwards to determine a series of supporting concepts to discuss a target concept.

This is only an initial step in developing encoding algorithms. Messages are created in a linear format, and conversion to a two-dimensional graphical format would require further development in symbology and spatial placement. To create more context that could help in decoding and understanding the message, the expression forms could be expanded. For example, the algorithms currently only use Planck unit expressions to describe the concepts of mass, length, and time using universal concepts. Other natural unit systems such as Stoney or Atomic units could also be used to provide redundancy and improve the chance of understanding. Atomic units are especially interesting because their equations can define units for length, mass, time, and charge using only two new terms: the reduced planck constant and the speed of light (assuming the mass of an electron and the fine structure constants were defined earlier with ratios). Using Planck units, three new terms are needed: the reduced planck constant, the speed of light, and the gravitational constant. This work uses infix notation (as opposed to prefix or postfix) and uses base ten, but it could easily be modifed for other options. It is hoped that this effort is the start of improving message encoding from an art done by individuals to a science done with algorithms.

5. Acknowledgements

The author would like to thank Doug Vakoch and Carl DeVito for discussions about this paper.

References

[1] M. Matessa, D. Vakoch, T. Lower, J. Barkow, C. DeVito, How to decode interstellar messages, Acta Astronautica 192 (2022) 77–81.

[2] B. McConnell, The Alien Communication Handbook, Springer, 2021.

[3] M. Matessa, Algorithms for decoding interstellar messages, Journal of the British Interplanetary Society 75 (10) (2022) 357–361.

[4] H. Freudenthal, LINCOS: Design of a language for cosmic intercourse, Amsterdam: North-Holland Publishing Company, 1960.

[5] M. Busch, R. Reddick, Testing SETI message designs, arXiv preprint arXiv:0911.3976 (2009).

[6] D. Vakoch, M. Matessa, C. DeVito, A. Kaiser, A mathematical and scientific message for active SETI, 42nd COSPAR Scientific Assembly 42 (2018) F3–8.

[7] F. D. C. Sagan, The Arecibo message of November, 1974, Icarus 26 (4) (1975) 462–466.

[8] Y. Dutil, S. Dumas, Error correction scheme in active SETI, IAA Publication IAA-01-IAA 9 (09) (2001).

[9] R. Heller, Decryption of messages from extraterrestrial intelligence using the power of social media–the seti decrypt challenge, International Journal of Astrobiology 18 (4) (2019) 296–303.

[10] D. De Paulis, P. Ferri, S. Shostak, D. Estévez, S. Montebugnoli, G. Bianchi, A. Melis, D. Scuka, A sign in space: An interdisciplinary exploration of the potential reception of an extraterrestrial signal, Acta Astronautica 212 (2023) 436–440.

[11] A. Ollongren, Astrolinguistics: Design of a Linguistic System for Interstellar Communication Based on Logic, Springer Science & Business Media, 2012.

[12] A. Ollongren, D. Vakoch, Typing logic contents using Lingua Cosmica, Acta Astronautica 68 (3-4) (2011) 535–538.

[13] P. Fitzpatrick, Outline of Cosmic OS, https://people.csail.mit.edu/paulfitz/cosmicos.shtml, retrieved 2024-4-17 (2003).

[14] P. Fitzpatrick, Cosmic OS, https://cosmicos.github.io/, retrieved 2024-4-17 (2014).

Appendix A.

```
(1 = 1)
(2 = 2)
(1 = [ o ])
(2 = [ o o ])
(3 = [ o o o ])
(4 = [ o o o o ])
(5 = [ o o o o o ])
(6 = [ o o o o o o ])
(7 = [ o o o o o o o ])
(8 = [ o o o o o o o o ])
(9 = [ o o o o o o o o o ])
(0 = [ ])
(9 - 4 = 5)
(4 - 9 = - 5)
(4 + 5 = 9)
(6 / 2 = 3)
(2 * 3 = 6)
(SQRT [ 9 ] = 3)
(2 ^ 3 = 8)
(PLANCK [ LENGTH ] = SQRT [ [ HBAR * G ] / [ C ^ 3 ] ])
(PLANCK [ MASS ] = SQRT [ [ HBAR * C ] / G ])
(PLANCK [ TIME ] = SQRT [ [ HBAR * G ] / [ C ^ 5 ] ])
```

Appendix B.

```
(S1 S2 S1)
(S3 S2 S3)
(S1 S2 S4 S5 S6)
(S3 S2 S4 S5 S5 S6)
(S7 S2 S4 S5 S5 S5 S6)
(S8 S2 S4 S5 S5 S5 S5 S6)
(S9 S2 S4 S5 S5 S5 S5 S5 S6)
(S10 S2 S4 S5 S5 S5 S5 S5 S5 S6)
(S11 S2 S4 S5 S5 S5 S5 S5 S5 S5 S6)
(S12 S2 S4 S5 S5 S5 S5 S5 S5 S5 S5 S6)
(S13 S2 S4 S5 S5 S5 S5 S5 S5 S5 S5 S5 S6)
(S14 S2 S4 S6)
(S13 S15 S8 S2 S9)
(S8 S15 S13 S2 S15 S9)
(S8 S16 S9 S2 S13)
(S10 S17 S3 S2 S7)
(S3 S18 S7 S2 S10)
(S19 S4 S13 S6 S2 S7)
(S3 S20 S7 S2 S12)
(S21 S4 S22 S6 S2 S19 S4 S4 S23 S18 S24 S6 S17 S4 S25 S20 S7 S6 S6)
(S21 S4 S26 S6 S2 S19 S4 S4 S23 S18 S25 S6 S17 S24 S6)
(S21 S4 S27 S6 S2 S19 S4 S4 S23 S18 S24 S6 S17 S4 S25 S20 S9 S6 S6)
```

Appendix C.

```
(S1 1)
(S2 =)
(S3 2)
(S4 [)
(S5 o)
(S6 ])
(S7 3)
(S8 4)
(S9 5)
(S10 6)
(S11 7)
(S12 8)
(S13 9)
(S14 0)
(S15 -)
(S16 +)
(S17 /)
(S18 *)
(S19 SQRT)
(S20 ^)
(S21 PLANCK)
(S22 LENGTH)
(S23 HBAR)
(S24 G)
(S25 C)
(S26 MASS)
(S27 TIME)
```